



Strathmore University

@iLabAfrica Centre

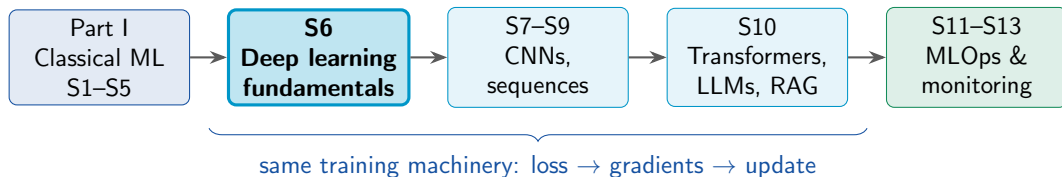
Session 6: Deep Learning Fundamentals

Javier Serrano

Applied Machine Learning

Master in Data Science & Analytics

Where this session sits in the course



Takeaway: Everything you learn today about training reappears, unchanged, in every later session.

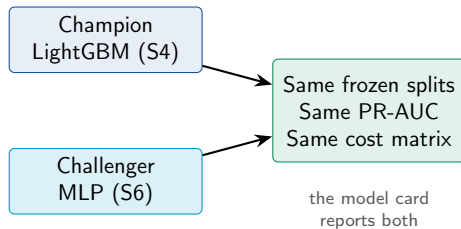
Learning outcomes

By the end of this session you will be able to:

1. Explain the anatomy of an MLP and compute a forward pass by hand.
2. Derive the training loop: loss, gradient descent, backpropagation.
3. Diagnose the classic failure modes and apply the right fix.
4. Configure modern optimisers and regularisers with justification.
5. Build and train an MLP in **TensorFlow/Keras**, with embeddings.
6. Judge when deep learning beats gradient boosting on tabular data.

The running case study

- Nairobi digital lender, $\approx 3.4\text{M}$ mobile-money wallets.
- Labelled loan book with $\approx 4\%$ default rate.
- Asymmetric costs: KES 10,000 per missed default vs. KES 800 per wrongly rejected borrower.
- Session 4 champion: a tuned LightGBM.



Takeaway: Today we build the *challenger*, not a replacement.

Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

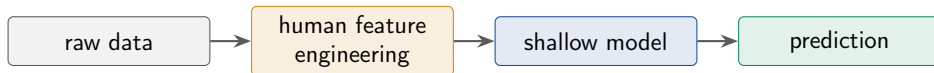
Making Training Work in Practice

TensorFlow and Keras in Practice

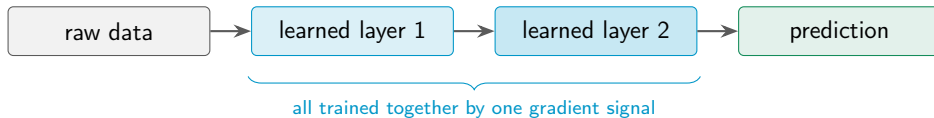
Lab

Two ways to get good features

Sessions 2–4: you engineer the features



Session 6 onwards: the network learns them

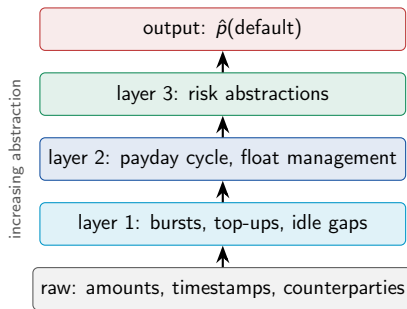


Depth builds a hierarchy of abstractions

- Each layer is a simple transformation:

$$h^{(l)} = \phi\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

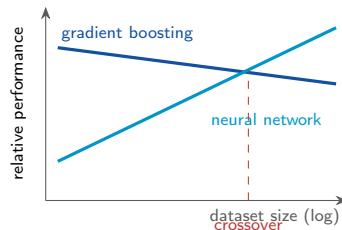
- Stacking them composes meaning: local patterns $\rightarrow$ behaviours $\rightarrow$ risk.
- No layer is told what to represent; the loss decides.



When gradient boosting still wins

- $n \lesssim 10^5$ rows of purely tabular, i.i.d. data.
- Heterogeneous features that are already informative.
- Limited training budget and MLOps maturity.
- You need SHAP explanations tomorrow morning.

Grinsztajn, Oyallon & Varoquaux (NeurIPS 2022) show tree ensembles still lead on medium-sized tabular benchmarks.



Schematic, not measured: the crossover point depends on your data, not on fashion.

When a neural network earns its keep

- High-cardinality IDs (120k agents) → learned embeddings.
- Multi-modal inputs: transactions + SMS text + geospatial.
- Shared representations across several lending products.
- Warm-start updates instead of nightly full retraining.

Criterion	LGBM	MLP
Small tabular data	strong	weak
ID embeddings	limited	native
Multi-modal fusion	poor	natural
Training cost	low	higher
Incremental update	retrain	warm start
Interpretability	easier	harder

Takeaway: Benchmark honestly; CBK model-risk expectations assume a documented challenger comparison.

Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

Making Training Work in Practice

TensorFlow and Keras in Practice

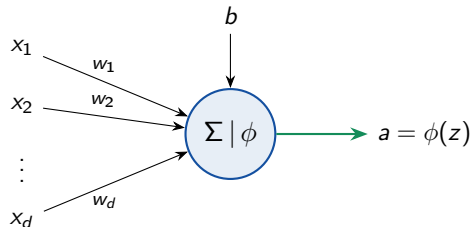
Lab

The artificial neuron

- Weighted sum, bias, non-linear activation:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \phi(z)$$

- w says *what matters*; b shifts the threshold; ϕ bends the response.
- One neuron draws one linear boundary.



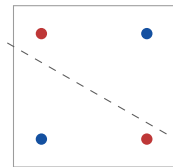
In our case study: x_1 = 90-day transaction count, x_2 = average float, a = evidence of repayment capacity.

Why the non-linearity is not optional

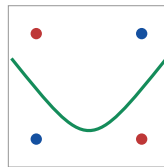
- Without ϕ , two stacked layers collapse:

$$W^{(2)}(W^{(1)}x) = \underbrace{(W^{(2)}W^{(1)})}_\text{one matrix} x$$

- A hundred linear layers still describe one linear model.
- The step-function perceptron (Rosenblatt, 1957) cannot solve XOR and is not differentiable.



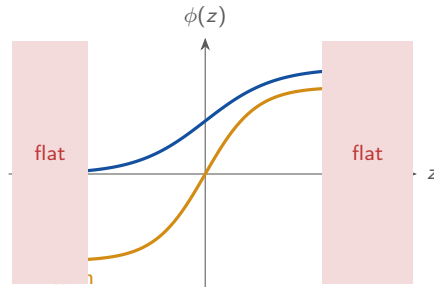
XOR: no straight line separates the classes



with ϕ : bent boundaries become available

Saturating activations: sigmoid and tanh

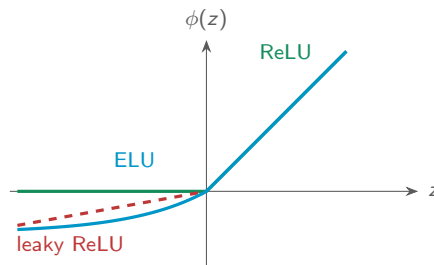
- $\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$ the natural **output** unit for a probability.
- $\tanh(z) = 2\sigma(2z) - 1 \in (-1, 1)$ zero-centred, historically used in hidden layers.
- Both **saturate**: for $|z|$ large the slope is ≈ 0 , so gradients die.



Takeaway: $\sigma'(z) \leq 0.25$ everywhere a built-in gradient shrinker.

The ReLU family: today's default

- $\text{ReLU}(z) = \max(0, z)$: cheap, sparse, no saturation for $z > 0$.
- Risk: a unit stuck at $z < 0$ never recovers a **dead unit**.
- Leaky ReLU / ELU keep a small negative slope; GELU is the Transformer default (Session 10).



Takeaway: ReLU in hidden layers, sigmoid or softmax at the output.

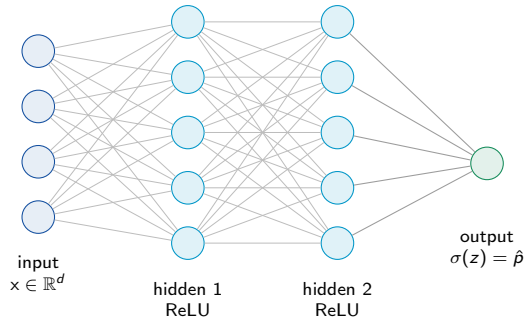
The Multi-Layer Perceptron

- Dense layers: every unit sees every unit of the previous layer.
- Forward pass, $l = 1, \dots, L$:

$$h^{(l)} = \phi\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

with $h^{(0)} = x$.

- Output layer chosen by the task, not by taste.



Count your parameters before you train

- A dense layer holds $(\text{fan-in} + 1) \times \text{fan-out}$ parameters.
- With 40 scaled features and a 64–32 network:

Layer	Shape	Params
Dense 1	40 $\rightarrow$ 64	2,624
Dense 2	64 $\rightarrow$ 32	2,080
Output	32 $\rightarrow$ 1	33
Total		4,737

- **Universal approximation:** one sufficiently wide hidden layer can approximate any continuous function.
- In practice depth is far cheaper than width for the same expressiveness.
- Rule of thumb: if parameters $\gg$ training rows, expect to fight overfitting all afternoon.

Takeaway: `model.summary()` is the first thing you read, not the last.

A forward pass by hand

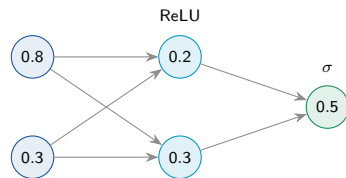
Two features, one hidden layer of two ReLU units:

$$\mathbf{x} = \begin{pmatrix} 0.8 \\ 0.3 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -0.4 \end{pmatrix}$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & -2 \\ 0.5 & 1 \end{pmatrix} \Rightarrow \mathbf{z}^{(1)} = \mathbf{h}^{(1)} = \begin{pmatrix} 0.2 \\ 0.3 \end{pmatrix}$$

With $\mathbf{w}^{(2)} = (2, -1)^\top$ and $b^{(2)} = -0.1$:

$$\mathbf{z}^{(2)} = 0.0 \Rightarrow \hat{p} = \sigma(0.0) = 0.5$$



Takeaway: An untrained network sits near $\hat{p} = 0.5$; that is the number you must beat.

The same forward pass in TensorFlow

```
import tensorflow as tf

borrower
x = tf.constant([[0.8, 0.3]])           # shape (1, 2): one
W1 = tf.constant([[1.0, 0.5], [-2.0, 1.0]]) # shape (2, 2)
b1 = tf.constant([0.0, -0.4])
w2 = tf.constant([2.0], [-1.0])
b2 = tf.constant([-0.1])

h = tf.nn.relu(x @ W1 + b1)             # hidden layer
p = tf.sigmoid(h @ w2 + b2)             # P(default)
print(p.numpy())                        # [[0.5]]
```

Keras stores weights so the pass reads $x @ W + b$ with row-vector inputs. When a model misbehaves, check shapes before you question the mathematics.

Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

Training: Loss, Gradients, Backpropagation

Making Training Work in Practice

TensorFlow and Keras in Practice

Lab

The loss turns "good model" into a number

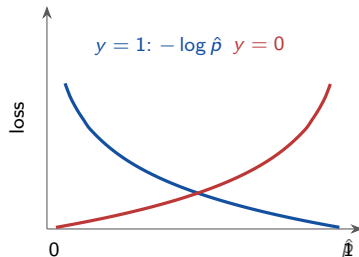
- Training minimises the average loss:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i))$$

- **Binary cross-entropy** for our default flag:

$$\mathcal{L}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

- It is the negative log-likelihood of a Bernoulli model: confident and wrong is punished without mercy.



Choosing the loss and the output layer together

Task	Output layer	Loss
Binary default flag	1 unit, sigmoid	binary cross-entropy
Loan purpose, K classes	K units, softmax	categorical cross-entropy
Recovery amount (KES)	1 unit, linear	MSE (or Huber if outliers)
Days until repayment	1 unit, softplus	Poisson / MSE on log

- Imbalance carries over from Session 3: pass `class_weight` to `fit`, or use focal loss for extreme skew.
- Never optimise or report plain accuracy on a 4% default rate.

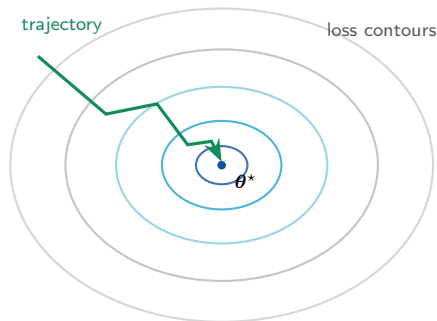
Looking ahead: Session 10: LLM pre-training is this same cross-entropy, applied to next-token prediction.

Gradient descent

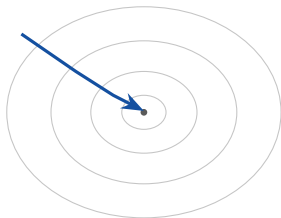
- Step downhill against the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

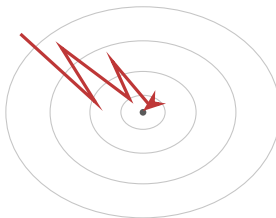
- The gradient points uphill; the minus sign is the whole algorithm.
- η (learning rate) sets the step length.



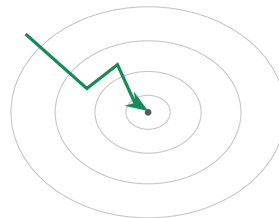
Batch, stochastic, and mini-batch



Batch: all n rows
exact, slow



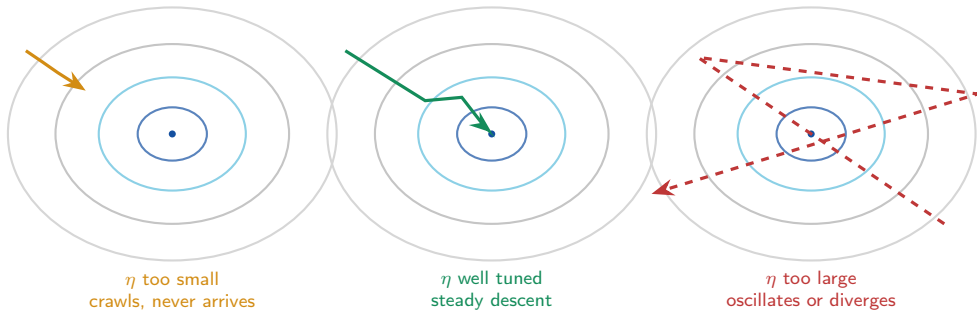
Stochastic: 1 row
noisy, cheap



Mini-batch: 32–512
the default

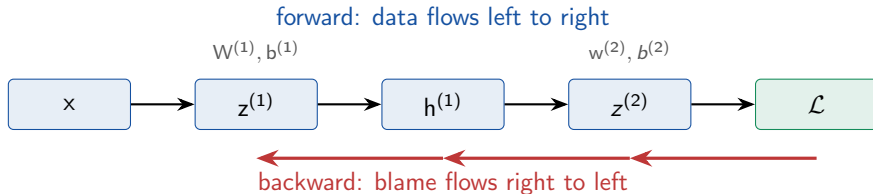
Takeaway: Mini-batch noise is a feature: it regularises, and it keeps the GPU busy.

The learning rate is the hyperparameter that matters



Takeaway: Tune η on a log grid before you touch anything else in the architecture.

Backpropagation: the idea



- One backward sweep produces every partial derivative, at roughly the cost of two forward passes.
- That efficiency is the reason networks with millions of parameters are trainable at all.

Backpropagation: the equations

Write $\delta^{(l)} = \partial \mathcal{L} / \partial z^{(l)}$. Then blame propagates as

$$\delta^{(l)} = \left(W^{(l+1)\top} \delta^{(l+1)} \right) \odot \phi' \left(z^{(l)} \right)$$

and the parameter gradients are

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} h^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

- Sigmoid output with cross-entropy gives the clean starting point

$$\delta^{(\text{out})} = \hat{p} - y$$

- Note the two multiplicative factors: the weights and the activation slope. Both will matter on the next slide.

Takeaway: You derive this once; TensorFlow does it forever after.

Autodiff in TensorFlow: `tf.GradientTape`

ops

```
W = tf.Variable(tf.random.normal([n_features, 1]) * 0.1)
b = tf.Variable(tf.zeros([1]))
opt = tf.keras.optimizers.SGD(learning_rate=0.1)

for step in range(500):
    with tf.GradientTape() as tape:                                # records the forward

        logits = X_train @ W + b
        loss = tf.reduce_mean(
            tf.nn.sigmoid_cross_entropy_with_logits(y_train, logits))
        grads = tape.gradient(loss, [W, b])                      # one backward sweep
        opt.apply_gradients(zip(grads, [W, b]))                  # parameter update
```

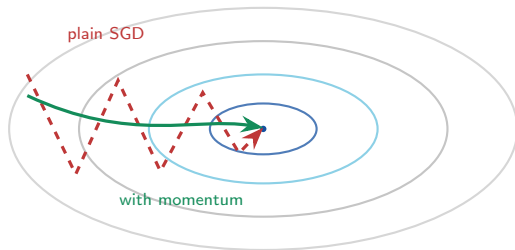
Pass **logits** to `..._with_logits` losses: it is numerically stable and avoids $\log 0$. We write this loop by hand in the lab so that `model.fit` is never a black box.

Momentum: a heavy ball down a narrow valley

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\theta} J$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$$

- Averages recent gradients, cancelling the zig-zag across the valley.
- Typical $\beta = 0.9$.



Adam: a per-parameter step size

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

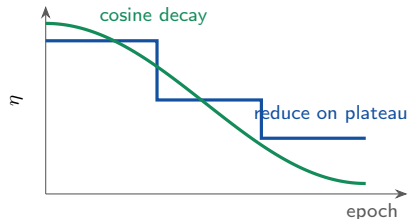
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

with bias corrections $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$ and $\hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$.

- $\mathbf{m}$ is momentum; $\mathbf{v}$ tracks gradient magnitude per parameter.
- Rarely-updated weights (embeddings!) get larger effective steps.
- **Defaults that work:** $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.
- AdamW decouples weight decay the Transformer standard (Session 10).

Learning-rate schedules

- Large steps early to explore, small steps late to settle.
- **Reduce on plateau:** halve η when validation loss stalls simple and effective.
- **Cosine decay** with warm-up: the deep-learning default for large models.



Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

Making Training Work in Practice

TensorFlow and Keras in Practice

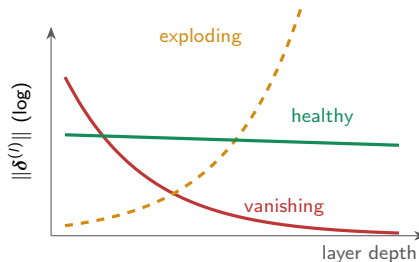
Lab

Vanishing and exploding gradients

- Backprop *multiplies* factors across layers:

$$\delta^{(1)} \propto \prod_l W^{(l)\top} \text{diag}(\phi'(z^{(l)}))$$

- Factors below 1 $\Rightarrow$ signal vanishes; above 1 $\Rightarrow$ it explodes.
- Deep sigmoid stacks vanish by construction, since $\sigma' \leq 0.25$.



Initialisation keeps the signal alive

- All-zero weights: every unit computes the same thing forever. Symmetry must be broken.
- Too large: activations saturate; too small: signal dies within three layers.
- Scale the variance to the fan-in:

$$\text{Glorot: } \text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

$$\text{He (ReLU): } \text{Var}(w) = \frac{2}{n_{\text{in}}}$$

Activation	Initialiser
ReLU, leaky ReLU	he_normal
tanh, sigmoid	glorot_uniform
SELU	lecun_normal

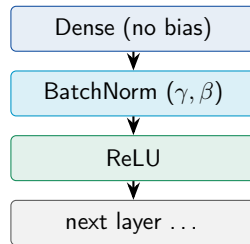
Takeaway: Other remedies for exploding gradients: gradient clipping now, skip connections in Session 7.

Batch normalisation

- Normalise pre-activations across the mini-batch, then let the network rescale:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \tilde{z} = \gamma \hat{z} + \beta$$

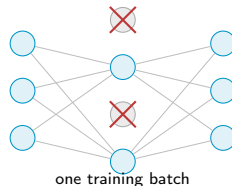
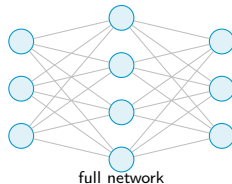
- γ, β are learned, so nothing is lost.
- Stabilises gradients, permits larger η , regularises mildly.



Deployment trap: at inference BatchNorm uses moving averages, not batch statistics. Keras handles this via the `training` flag hand-rolled serving code often does not (Session 11).

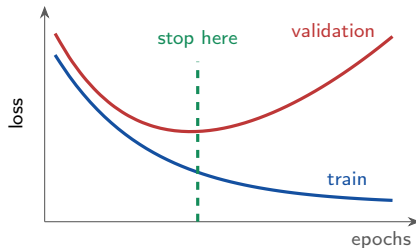
Dropout: an ensemble hiding inside one network

- During training, drop each unit with probability p (0.2–0.5).
- Every batch trains a different thinned network; inference averages them.
- This is Session 4's bagging intuition, at the level of units.



Early stopping and weight decay

- **Early stopping:** watch the validation metric, stop after patience epochs without improvement, restore the best weights.
- It turns "number of epochs" into a decision the data makes for you.
- **Weight decay (ℓ_2):** add $\lambda \|\theta\|_2^2$ to the loss Ridge shrinkage, as in Session 3.



Scale your inputs, or the optimiser suffers

- KES amounts in millions beside ratios in $[0, 1]$ produce a stretched, ill-conditioned loss surface.
- Standardise: $x' = (x - \mu)/\sigma$, with statistics from the **training fold only**.
- Trees did not care about this. Gradient descent does.



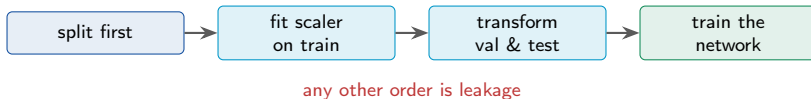
unscaled features:
zig-zag, slow



standardised:
straight to the minimum

The rest of the data hygiene checklist

- **Missing values:** networks cannot ingest NaN. Impute, and add a missingness indicator column (Session 2 pipeline).
- **Shuffle** training rows each epoch so mini-batches are representative but keep the temporal validation split intact.
- **Categoricals:** one-hot for low cardinality, embeddings for high cardinality (two slides from now).
- **Reproducibility:** `tf.keras.utils.set_random_seed(42)`, pinned library versions, logged data window.



Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

Training: Loss, Gradients, Backpropagation

Making Training Work in Practice

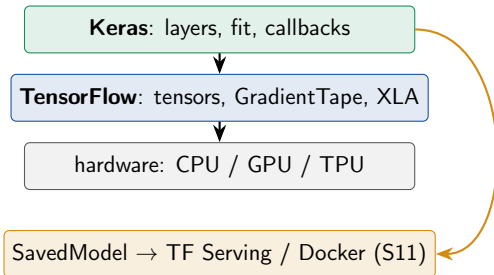
TensorFlow and Keras in Practice

Lab

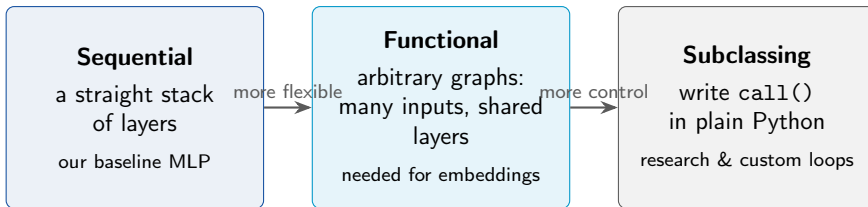
The TensorFlow stack

- **TensorFlow core:** tensors, autodiff, XLA, CPU/GPU/TPU.
- **Keras:** layers, losses, optimisers, metrics, callbacks.
- **SavedModel** is the artefact that travels to production.

PyTorch is the research-side counterpart; concepts transfer one to one. This course standardises on TensorFlow end to end.



Three ways to define a model



Takeaway: Start Sequential; move to Functional the moment you have two input branches.

Your first Keras model

```
import tensorflow as tf
from tensorflow.keras import layers

tf.keras.utils.set_random_seed(42)

model = tf.keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(64, activation="relu", kernel_initializer="he_normal"),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu", kernel_initializer="he_normal"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),
])
model.summary()
```

parameter count

Compiling: optimiser, loss, and the right metrics

```
model.compile(  
optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
loss="binary_crossentropy",  
metrics=[  
tf.keras.metrics.AUC(name="roc_auc"),  
tf.keras.metrics.AUC(curve="PR", name="pr_auc"),    # the one that  
matters  
tf.keras.metrics.Recall(name="recall"),  
],  
)
```

- With a 4% default rate, accuracy is 96% for a model that approves everyone.
- PR-AUC and recall at a fixed alert budget are the honest numbers (Session 3).

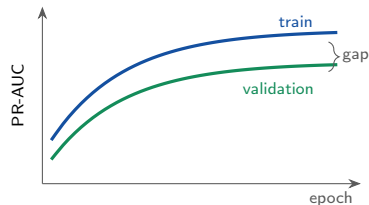
Training with callbacks

ceiling

```
callbacks = [  
    tf.keras.callbacks.EarlyStopping(monitor="val_pr_auc", mode="max",  
    patience=10, restore_best_weights=True),  
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,  
    patience=4),  
    tf.keras.callbacks.TensorBoard(log_dir="logs/mlp_v1"),  
]  
  
history = model.fit(  
    X_train, y_train, validation_data=(X_val, y_val),  
    epochs=200, batch_size=256, # epochs is only a  
  
    class_weight={0: 1.0, 1: pos_weight}, # ~4% default rate  
    callbacks=callbacks, verbose=2,  
)
```

Reading the training curves

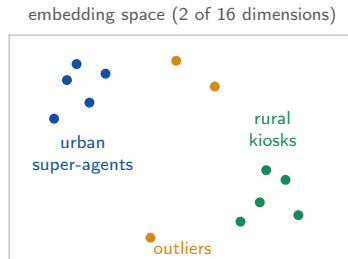
Curve shape	Reading
both losses fall, gap small	healthy; train longer
train falls, val rises	overfitting
both flat and high	underfitting or η too small
loss jumps to NaN val very jagged	η too high; clip batch too small



Takeaway: TensorBoard is your first stop, not a screenshot at the end.

Entity embeddings: dense vectors instead of 120k columns

- One-hot for 120,000 agents is a wall of sparse columns.
- An embedding learns a vector in $\mathbb{R}^{16}$ per agent, trained by the same gradient.
- Similar agents end up close together, a learned geography of risk.



Embeddings with the Functional API

```
num_in    = layers.Input(shape=(n_features,), name="numeric")
agent_in  = layers.Input(shape=(), dtype="int64", name="agent_id")

emb = layers.Embedding(input_dim=n_agents + 1,          # index 0
                        reserved for unseen
                        output_dim=16)(agent_in)        # one 16-d vector per agent
emb = layers.Flatten()(emb)

x      = layers.Concatenate()([num_in, emb])
x      = layers.Dense(64, activation="relu")(x)
x      = layers.Dropout(0.3)(x)
out    = layers.Dense(1, activation="sigmoid")(x)

model = tf.keras.Model(inputs=[num_in, agent_in], outputs=out)
```

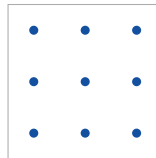
Production rule: agents unseen at training time must map to index 0, not crash the service (Session 13).

Searching hyperparameters, in priority order

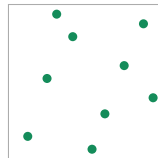
1. Learning rate: $[10^{-4}, 10^{-2}]$, log scale.
2. Width $\{32, 64, 128, 256\}$ and depth $\{1, 2, 3\}$.
3. Dropout $\{0.0, 0.2, 0.3, 0.5\}$.
4. Batch size $\{128, 256, 512\}$.

Random or Bayesian search beats grids (Session 4).

Tools: Keras Tuner with Hyperband, or Optuna wrapping a Keras objective.



grid: 3 distinct
learning rates



random: 9 distinct
learning rates

Takeaway: Many short trials with early stopping beat a few long ones.

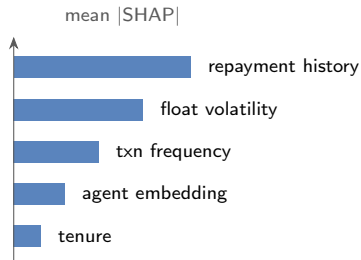
Debugging triage table

Symptom	Likely cause	First fix
Loss becomes NaN	η too high, log 0	lower η , use logits, clip
Loss frozen at ≈ 0.693	dead ReLUs, bad init	He init, lower η
Train PR-AUC $\gg$ validation	overfitting	dropout, early stopping
Validation metric jumps wildly	batch too small	larger batch, BatchNorm
Painfully slow convergence	unscaled inputs	standardise features
Works in notebook, fails in API	BatchNorm / scaler mismatch	export the full pipeline

This table ships as a markdown cell in the lab notebook. Keep it beside you during the capstone.

Explaining a network to a credit committee

- A declined micro-loan needs a reason, and "the network said so" is not one.
- Use `shap.DeepExplainer` or integrated gradients for per-decision attributions.
- Report global attributions in the model card, next to the LightGBM ones.



Fairness, calibration, and compute realism

- **Segment your evaluation:** PR-AUC and calibration per county, gender, and device tier. A network can quietly trade rural recall for urban precision.
- **Regulatory frame:** the Kenya Data Protection Act (2019) and the CBK Digital Credit Providers Regulations (2022) make segment testing and documented reasoning an obligation.
- **Calibration:** deep networks are often over-confident. Check a reliability curve and apply temperature scaling before probabilities reach a pricing engine.
- **Compute realism:** these MLPs train in minutes on CPU. Save GPU budget for Sessions 7–10; efficiency is a virtue in low-margin lending.

Roadmap

From Classical ML to Deep Learning

Anatomy of a Neural Network

Training: Loss, Gradients, Backpropagation

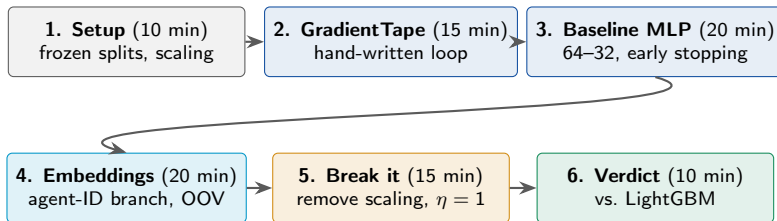
Making Training Work in Practice

TensorFlow and Keras in Practice

Lab

Lab 6 (60 minutes): the MLP challenger

Brief: the risk committee has asked whether a neural network with agent embeddings can beat the Session 4 LightGBM. You build the challenger and report honestly.

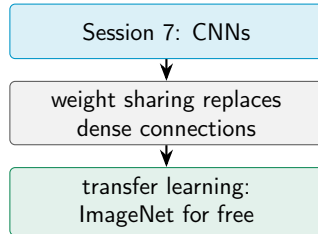


Summary

- An MLP is stacked linear maps with non-linear activations; depth learns representations you would otherwise hand-craft.
- Training is loss $\rightarrow$ gradients by backpropagation $\rightarrow$ update by SGD or Adam. Nothing more.
- Stability comes from He initialisation, ReLU, batch normalisation, and scaled inputs.
- Generalisation comes from dropout, early stopping, and weight decay.
- Keras gives you Sequential for stacks and Functional for embeddings; callbacks do the tedious work.
- Gradient boosting remains the tabular champion to beat, so beat it, or report that you did not.

Before the next session

- Read chapter 6 of the book.
- (optional) Read Géron, chapters 10–11.
- (Optional) Skim Prince, *Understanding Deep Learning*, chapters 3–7 for the theory view.
- Assignment 3
- Capstone project: start to think about the proposals (they are due at the mid-semester break).



References

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. Chapters 10–11.
- Bishop, C. M., & Bishop, H. (2024). *Deep Learning: Foundations and Concepts*.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press. Chapters 3–7.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
- Ioffe, S., & Szegedy, C. (2015). Batch Normalization. *ICML*.
- Srivastava, N., et al. (2014). Dropout. *JMLR*, 15.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *NeurIPS*.
- Guo, C., & Berkhahn, F. (2016). Entity Embeddings of Categorical Variables. *arXiv:1604.06737*.